

Your grade: 100%

Your latest: 100% • Your highest: 100% • To pass you need at least 80%. We keep your highest score.

Next item →

1.

When profiling your program, should you expect it to run more slowly than it would normally?

1 / 1 point
- ☒ Yes; the profiler modifies the program in order to measure where it spends the most time

☐ No; if the profiler affected the program's speed, the data collected by the profiler would be inaccurate, so the profiler runs in the background and monitors the program as it runs

✔ Nice work

Correct. A profiler is measuring the relative computational cost of various parts of the code.
2.

Of the methods below, which is generally the most efficient at achieving program speedup, and why?

1 / 1 point
- ☐ Writing your own mathematical functions and routines for your program to utilize, because they are faster than published mathematical libraries

☒ Using published mathematical libraries, because they're optimized for speed and efficiency

☐ Writing your own mathematical functions and routines for your program to utilize, because you can tailor them to your particular situation better than published libraries

☐ Using published mathematical libraries, because they're often more reliable and bug-free than functions you could write yourself

✔ Nice work

Correct. Published libraries often already contain the performance-enhancing techniques we've already covered, so that you don't have to focus on implementing them.
3.

When using compiler optimization options, what trade-offs must be kept in mind?

1 / 1 point
- ☐ Certain compiler options can increase speed and efficiency, but may decrease accuracy

☐ Certain compiler options can increase speed and efficiency, but may also increase memory usage

☐ Certain compiler options can increase speed and efficiency, but may also restrict your program to a particular CPU type or model

☒ All of the above

✔ Nice work

Correct.
4.

What is the compiler option that enables no optimization

1 / 1 point
- ✔ Nice work

This compiler options enables no compiler optimization - see the [gnu compiler documentation](#) ↗
5.

Does the compiler options `-Ofast` change the results in a floating point intensive code?

1 / 1 point
- ☒ Yes

☐ No

✔ Nice work

This option allows the compiler to rearrange the order of how floating point operations are executed. This leads to results that are not exactly bit by bit the same as non-optimized code
6.

What is the purpose of software profiling?

1 / 1 point
- ☒ To identify the bottlenecks or hotspots in a program

☐ To debug and fix errors in the code

☐ To parallelize the code for better performance

☐ To optimize memory usage

✔ Nice work

Correct. Software profiling analyzes the behavior of a program as it is running and provides a report on the functions or subroutines that take the most time or are called the most frequently.
7.

Which of the following is NOT a benefit of using optimized numerical libraries like BLAS, LAPACK, or Intel MKL?

1 / 1 point
- ☐ They are highly optimized and can provide significant performance improvements over self-written code

☐ They save development time by providing pre-compiled functions for common mathematical operations

☐ They can be linked with your code to improve performance

☒ They can be easily modified and customized to suit specific requirements

✔ Nice work

Correct. Numerical libraries like BLAS, LAPACK, and Intel MKL are pre-compiled and optimized for performance. While they provide significant speedups and save development time, they are typically not designed to be easily modified or customized by the user.
8.

Which of the following is a benefit of using shared libraries over static libraries?

1 / 1 point
- ☐ Shared libraries result in larger executable file sizes

☐ Shared libraries are more portable across different systems

☒ Shared libraries can be updated without recompiling the executable

☐ Shared libraries typically have better performance than static libraries

✔ Nice work

Correct. With shared (dynamic) libraries, only references to the linked functions are included in the executable. This allows the library files containing the necessary instructions to be updated or changed between program runs without having to recompile the executable.
9.

What is the purpose of the `-mtune` compiler flag in GCC?

1 / 1 point
- ☐ To enable specific vector instruction sets like AVX2 or FMA

☒ To optimize the code for a particular type of CPU

☐ To perform interprocedural optimization across multiple source files

☐ To disable compiler optimization completely

✔ Nice work

Correct. The `-mtune` flag in GCC allows the compiler to produce instructions that are tuned for best performance on a particular type of CPU, such as Intel Skylake or AMD EPYC processors.
10.

Which of the following statements about compiler optimization levels is true?

1 / 1 point
- ☐ Higher optimization levels always result in faster executable files

☐ Optimization level `-O0` specifies the highest level of optimization

☐ Optimization level `-Og` is recommended for creating production executables

☒ Optimization level `-O3` includes more optimization options than `-O2`

✔ Nice work

Correct. In GCC, the `-O` flag specifies the optimization level, with higher numbers (`-O1`, `-O2`, `-O3`) including progressively more optimization options. `-O3` includes the most optimization options, while `-O0` specifies no optimization.